RN SHETTY TRUST®

RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution Affiliated to VTU, Recognized by GOK, Approved by AICTE
(NAAC 'A+ Grade' Accredited, NBA Accredited (UG - CSE, ECE, ISE, EIE and EEE)
Channasandra, Dr. Vishnuvardhan Road, Bengaluru - 560 098
Ph:(080)28611880,28611881 URL: www.rnsit.ac.in

Department of Master of Computer Applications

Machine learning and Data analytics using Python Laboratory

(MMC201) (IPCC)

II Semester MCA

Lab Manual-2025

VISION

"Building RNSIT into a world- class institution."

MISSION

"To impart high quality education in Engineering and Technology and Management with a difference, enabling students to excel in their career."

Prepared By

JYOTHI T
Asst. Professor,
Dept. of MCA,
RNSIT

ROOPA H M
Asst. Professor,
Dept. of MCA,
RNSIT

Sl. No	Experiments
1	Implement and demonstrate the FIND-S Algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.
2	For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate- Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.
3	Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.
4	Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file Compute the accuracy of the classifier, considering few test data sets.
5	Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions.
6	Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.
7	Write a program to demonstrate Regression analysis with residual plots on a given data set.
8	Write a program to compute summary statistics such as mean, median, mode, standard deviation and variance the given different types of data.
9	Write a program to implement k-Means clustering algorithm to cluster the set of data stored in .CSV file

Lab 1 Program

1. Implement and demonstrate the FIND-S Algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.

```
]: import csv

def load_data(filename):
    with open(filename, 'r') as file:
        data = list(csv.reader(file))
        header = data[0]
        samples = data[1:]
    return header, samples

def find_s_algorithm(data):
    hypothesis = []
    for row in data:
        if row[-1].lower() == 'yes':
            hypothesis = row[:-1]
            break

    for row in data:
        if row[-1].lower() == 'yes':
            for i in range(len(hypothesis)):
                if row[i] != hypothesis[i]:
                    hypothesis[i] = '?'
    return hypothesis

def print_result(header, hypothesis):
    print("Most specific hypothesis:")
    for attr, value in zip(header[:-1], hypothesis):
        print(f"{attr}: {value}")

if __name__ == "__main__":
    filename = "training_data.csv"
    header, samples = load_data(filename)
    hypothesis = find_s_algorithm(samples)
    print_result(header, hypothesis)
```

Output:

Most specific hypothesis:
Sky : Sunny
AirTemp: Warm
Humidity: ?
Wind: Strong
Water: ?
Forecast: ?

Lab 2 program

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate- Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.

```
import pandas as pd
import numpy as np

# Step 1: Load data
df = pd.read_csv("training_data.csv")
data = df.values
X = data[:, :-1]  # All columns except the last one
y = data[:, -1]   # Last column

# Step 2: Initialize specific and general hypotheses
n_features = X.shape[1]
S = ['φ'] * n_features
G = [['?'] * n_features]

# Step 3: Candidate Elimination Algorithm
for i, instance in enumerate(X):
    if y[i] == "Yes":  # Positive example
        for j in range(n_features):
            if S[j] == 'φ':
                S[j] = instance[j]
            elif S[j] != instance[j]:
                S[j] = '?'
        # Remove hypotheses from G that are inconsistent with S
        G = [g for g in G if all(s == '?' or s == g[i] or g[i] == '?' for i, s in enumerate(S))]
    else:  # Negative example
        new_G = []
        for g in G:
            for j in range(n_features):
                if g[j] == '?':
                    if S[j] != '?':
                        g_new = g.copy()
                        g_new[j] = S[j]
                        new_G.append(g_new)
        G += new_G

# Remove overly specific/general hypotheses
G = [list(x) for x in set(tuple(g) for g in G) if all(g[i] == '?' or g[i] != S[i] for i in range(n_features))]

# Step 4: Output version space
print("Final Specific Hypothesis (S):", S)
print("\n Final General Hypotheses (G):\n", G)
```

Output:

Final Specific Hypothesis (S): ['Sunny', 'Warm', '?', 'Strong', '?', '?']

Final General Hypotheses (G):

[['?', '?', '?', '?', '?', 'Same'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', 'Warm', '?'], ['Sunny', '?', '?', '?', '?', '?']]

Lab 3 program

3. Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

# Dataset
data = {
    'Weather': ['Sunny','Sunny','Cloudy','Rain','Rain','Rain','Cloudy','Sunny',
                'Sunny','Rain','Sunny','Cloudy','Cloudy','Rain'],
    'Temperature': ['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild',
                    'Cool','Mild','Mild','Mild','Hot','Mild'],
    'Humidity': ['High','High','High','High','Normal','Normal','Normal','High',
                 'Normal','Normal','Normal','High','Normal','High'],
    'Wind': ['Weak','Strong','Weak','Weak','Weak','Strong','Strong',
             'Weak','Weak','Weak','Strong','Strong','Weak','Strong'],
    'Play': ['No','No','Yes','Yes','Yes','No','Yes','No',
             'Yes','Yes','Yes','Yes','Yes','No']
}

# Create DataFrame
df = pd.DataFrame(data)
print(df)
```

```
# Encode categorical variables
le_Weather = LabelEncoder()
le_temperature = LabelEncoder()
le_humidity = LabelEncoder()
le_wind = LabelEncoder()
le_play = LabelEncoder()

df['Weather'] = le_Weather.fit_transform(df['Weather'])
df['Temperature'] = le_temperature.fit_transform(df['Temperature'])
df['Humidity'] = le_humidity.fit_transform(df['Humidity'])
df['Wind'] = le_wind.fit_transform(df['Wind'])
df['Play'] = le_play.fit_transform(df['Play'])
print(df)

# Features and target
X = df[['Weather', 'Temperature', 'Humidity', 'Wind']]
y = df['Play']

# Build Decision Tree using ID3 (entropy)
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)
```

```
# Predict for a new sample
# Example: Weather=Sunny, Temperature=Cool, Humidity=High, Wind=Strong
sample = [[le_Weather.transform(['Sunny'])[0],
           le_temperature.transform(['Cool'])[0],
           le_humidity.transform(['High'])[0],
           le_wind.transform(['Strong'])[0]]]


prediction = model.predict(sample)
print("Play Prediction:", le_play.inverse_transform(prediction)[0])

# Visualize the Decision Tree
plt.figure(figsize=(18, 10))
plot_tree(model, feature_names=X.columns, class_names=le_play.classes_, filled=True)
plt.title("Decision Tree using ID3 (Entropy)")
plt.show()
```

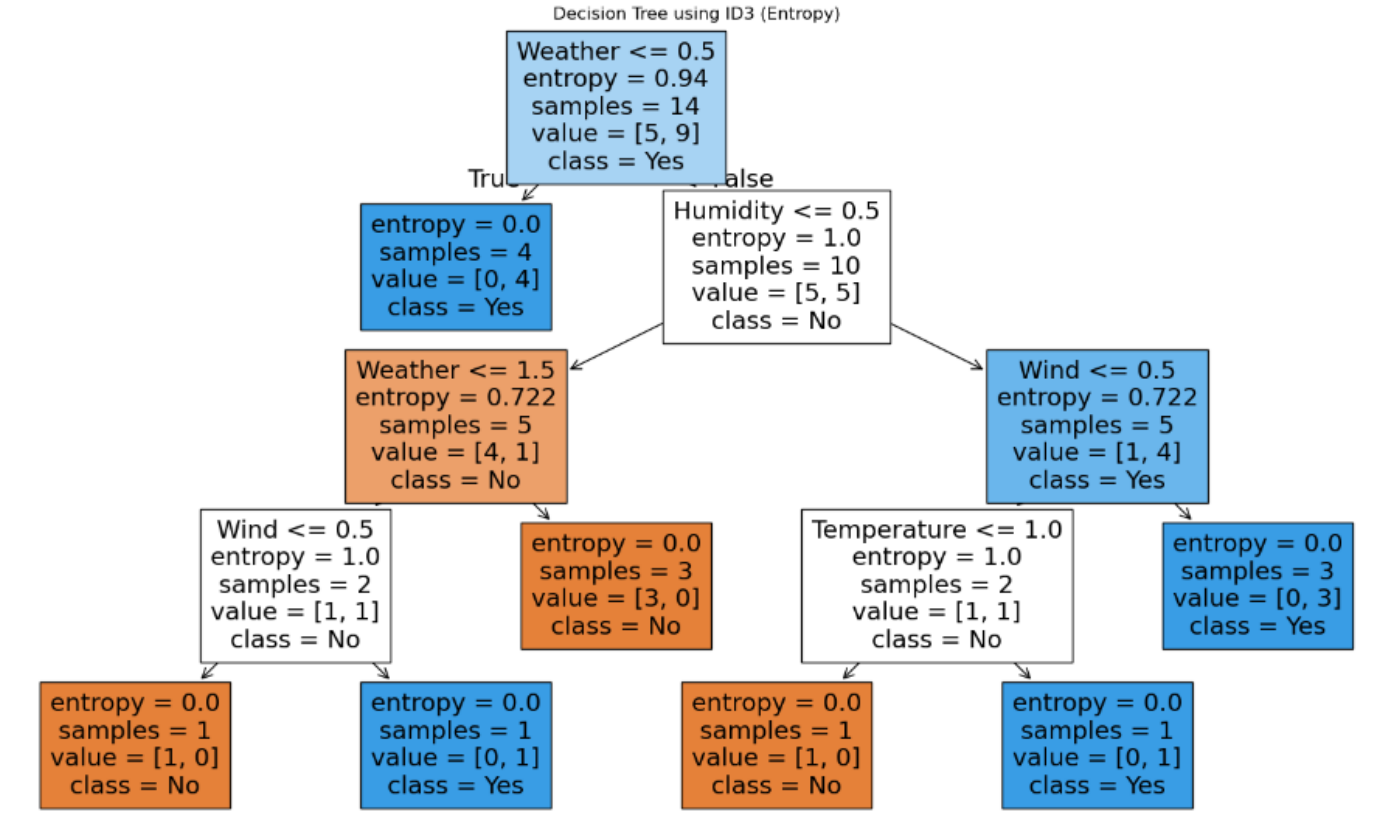
Output:

	Weather	Temperature	Humidity	Wind	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Cloudy	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Cloudy	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Cloudy	Mild	High	Strong	Yes
12	Cloudy	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

	Weather	Temperature	Humidity	Wind	Play
0	2	1	0	1	0
1	2	1	0	0	0
2	0	1	0	1	1
3	1	2	0	1	1
4	1	0	1	1	1
5	1	0	1	0	0
6	0	0	1	0	1
7	2	2	0	1	0
8	2	0	1	1	1
9	1	2	1	1	1
10	2	2	1	0	1
11	0	2	0	0	1
12	0	1	1	1	1
13	1	2	0	0	0

```
Play Prediction: No
```

```
Prediction: No
```

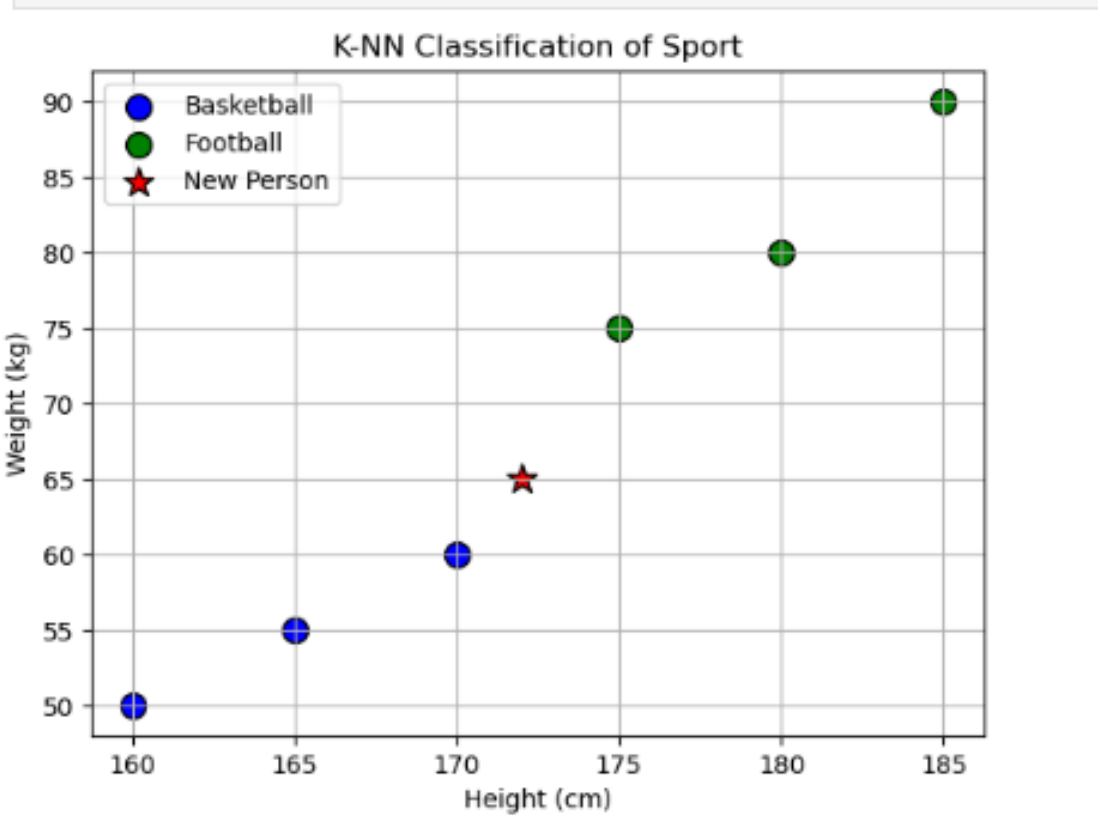


Lab 5 program

5.Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions.

```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

# Sample data
data = {
    'Height': [160, 165, 170, 175, 180, 185],
    'Weight': [50, 55, 60, 75, 80, 90],
    'Sport': ['Basketball', 'Basketball', 'Basketball', 'Football', 'Football', 'Football']
}

df = pd.DataFrame(data)

# Input features and labels
X = df[['Height', 'Weight']]
y = df['Sport']

# KNN model
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

# New data point to classify
new_person = [[172, 65]]
prediction = knn.predict(new_person)

# Plotting
colors = {'Basketball': 'blue', 'Football': 'green'}

# Plot original data points
for sport in df['Sport'].unique():
    subset = df[df['Sport'] == sport]
    plt.scatter(subset['Height'], subset['Weight'],
                c=colors[sport], label=sport, s=100, edgecolor='black')

# Plot the new point
plt.scatter(new_person[0][0], new_person[0][1],
            c='red', label='New Person', s=150, marker='*', edgecolor='black')

plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.title('K-NN Classification of Sport')
plt.legend()
plt.grid(True)
plt.show()
```

Output:




```
Predicted Sport: Basketball
```

Lab 7 program

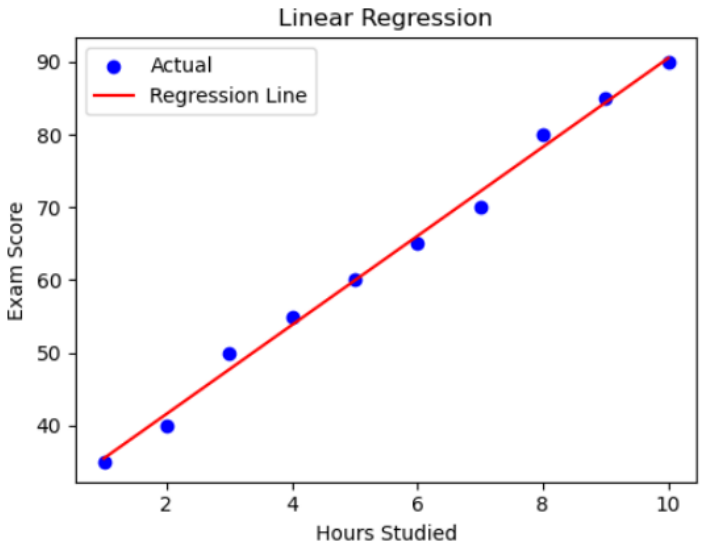
7. Write a program to demonstrate Regression analysis with residual plots on a given data set.

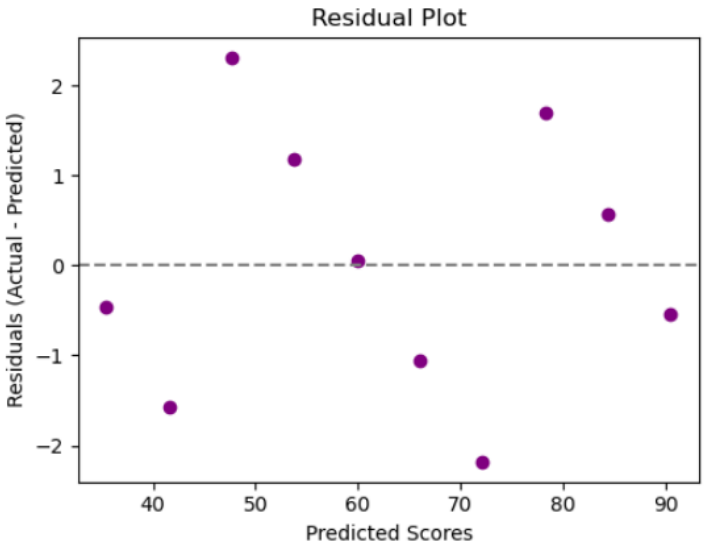
```python
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Sample data: Hours studied vs Exam score
data = {'Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'Scores': [35, 40, 50, 55, 60, 65, 70, 80, 85, 90]}

df = pd.DataFrame(data)

# Feature and target
X = df[['Hours']]
y = df['Scores']

# Train linear regression model
model = LinearRegression()
model.fit(X, y)

# Predictions
y_pred = model.predict(X)
```

```
# Residuals
residuals = y - y_pred

# Scatter plot + Regression line
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, y_pred, color='red', label='Regression Line')
plt.title('Linear Regression')
plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.legend()

# Residual plot
plt.subplot(1, 2, 2)
plt.scatter(y_pred, residuals, color='purple')
plt.axhline(y=0, color='gray', linestyle='--')
plt.title('Residual Plot')
plt.xlabel('Predicted Scores')
plt.ylabel('Residuals (Actual - Predicted)')

plt.tight_layout()
plt.show()
```

```
# Evaluation metrics
mae = mean_absolute_error(y, y_pred)
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)

print("Model Evaluation Metrics:")
print(f"MAE: {mae:.2f}")
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
```

Output:





Model Evaluation Metrics:

MAE: 1.16

MSE: 1.88

RMSE: 1.37

Lab 8 program

8.Write a program to compute summary statistics such as mean, median, mode, standard deviation and variance the given different types of data.

```
import pandas as pd

# Sample DataFrame
data = {
    'Math': [85, 90, 78, 92, 88],
    'Science': [80, 85, 84, 90, 87],
    'English': [75, 70, 72, 68, 74]
}

df = pd.DataFrame(data)

# Display the DataFrame
print("DataFrame:\n", df)

# Compute summary statistics
print("\n\nsummary statistics of the given data")
print("-----------------------------------------")
print(df.describe())
print("-----------------------------------------")

print("\n Mean:\n", df.mean())
print("\n Median:\n", df.median())
print("\n Mode:",df.mode())
#print("\n Mode:\n", df.mode().iloc[0])       # .iloc[0] to get one row if multiple modes

print("\n Standard Deviation:\n", df.std())
print("\n Variance:\n", df.var())
```

Output:

```
DataFrame:
    Math  Science  English
0    85       80       75
1    90       85       70
2    78       84       72
3    92       90       68
4    88       87       74


summary statistics of the given data
-----------------------------------------
            Math    Science    English
count   5.000000   5.000000   5.000000
mean   86.600000  85.200000  71.800000
std     5.458938   3.701351   2.863564
min    78.000000  80.000000  68.000000
25%    85.000000  84.000000  70.000000
50%    88.000000  85.000000  72.000000
75%    90.000000  87.000000  74.000000
max    92.000000  90.000000  75.000000
-----------------------------------------

 Mean:
 Math       86.6
Science    85.2
English    71.8
dtype: float64

 Median:
 Math       88.0
Science    85.0
English    72.0
dtype: float64
```

```
 Mode:    Math  Science  English
0    78        80       68
1    85        84       70
2    88        85       72
3    90        87       74
4    92        90       75

 Standard Deviation:
 Math       5.458938
Science    3.701351
English    2.863564
dtype: float64

 Variance:
 Math       29.8
Science    13.7
English     8.2
dtype: float64
```

Lab 9 Program

9. Write a program to implement k-Means clustering algorithm to cluster the set of data stored in .CSV file.

```
import warnings
warnings.filterwarnings("ignore")
```

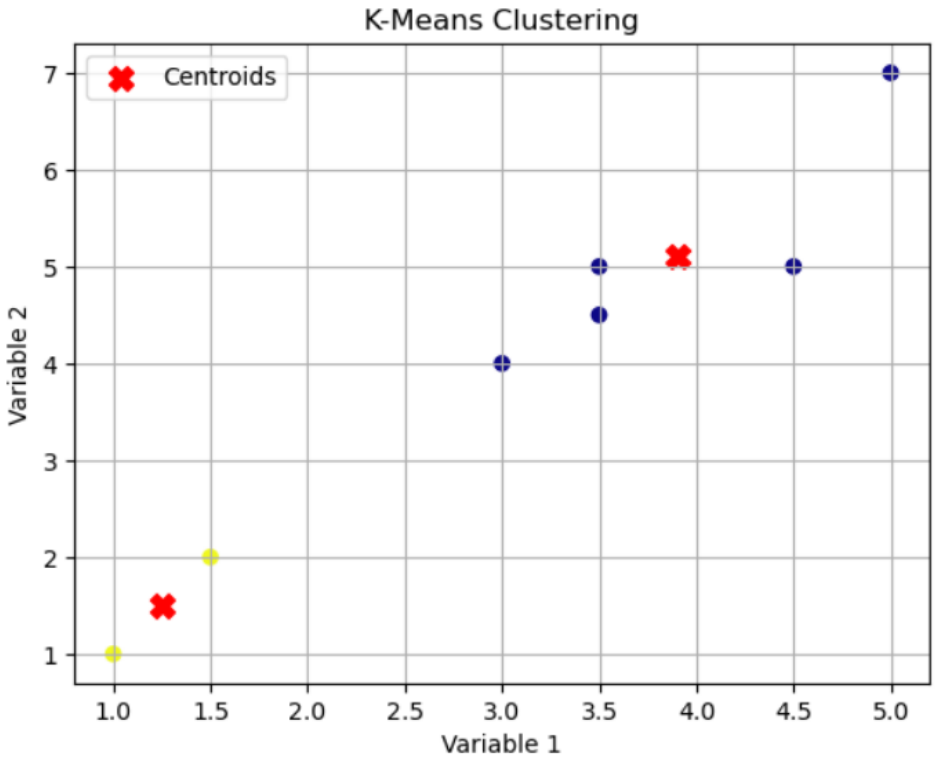
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Step 1: Load data (make sure it's only two columns)
data = pd.read_csv("data.csv")

# Step 2: Apply K-Means
kmeans = KMeans(n_clusters=2)
kmeans.fit(data)

# Step 3: Add cluster labels to the data
data['Cluster'] = kmeans.labels_
print(data)
```

```
# Step 4: Plot the clusters
plt.scatter(data.iloc[:, 0], data.iloc[:, 1], c=data['Cluster'], cmap='plasma')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=100, c='red', marker='X', label='Centroids')
plt.xlabel("Variable 1")
plt.ylabel("Variable 2")
plt.title("K-Means Clustering")
plt.legend()
plt.grid(True)
plt.show()
```

```
# Step 4: Plot the clusters
plt.scatter(data.iloc[:, 0], data.iloc[:, 1], c=data['Cluster'], cmap='plasma')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=100, c='red', marker='X', label='Centroids')
plt.xlabel("Variable 1")
plt.ylabel("Variable 2")
plt.title("K-Means Clustering")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
   Variable1  Variable2  Cluster
0        1.0        1.0        1
1        1.5        2.0        1
2        3.0        4.0        0
3        5.0        7.0        0
4        3.5        5.0        0
5        4.5        5.0        0
6        3.5        4.5        0
```



Lab 4 program

4 .Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file Compute the accuracy of the classifier, considering few test data sets.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

# Load CSV
data = pd.read_csv('naivejt.csv')

# Encode categorical features
le = LabelEncoder()
for column in data.columns:
    data[column] = le.fit_transform(data[column])

# Split features and target
X = data.drop('Play', axis=1)
y = data['Play']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Naive Bayes classifier
model = CategoricalNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

```
Accuracy: 100.00%
```

OR

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Load the Iris dataset
iris = load_iris()
X = iris.data  # Features
y = iris.target # Target variable (species)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a Gaussian Naive Bayes classifier
gnb = GaussianNB()

# Train the classifier on the training data
gnb.fit(X_train, y_train)

# Make predictions on the test data
y_pred = gnb.predict(X_test)

# Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=iris.target_names)

print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:")
print(report)
```

```
Accuracy: 0.98

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      0.92      0.96        13
```

Lab program 6

6. Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.

```
#simple python program on building an ann on backpropagation using dataset python
# Building a simple Artificial Neural Network (ANN) with Backpropagation using Python
#and NumPy

#This program demonstrates building and training a simple ANN using the backpropagation
#algorithm with
#Python and the NumPy library. It uses the XOR dataset to show the network's ability to
#learn non-linear relationships.

#python
import numpy as np

# 1. Define helper functions (activation, derivative, loss)

def sigmoid(x):
  """Sigmoid activation function."""
  return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
  """Derivative of the sigmoid function."""
  return x * (1 - x)

def mean_squared_error(y_pred, y_true):
  """Mean Squared Error loss function."""
  return np.mean(np.square(y_true - y_pred))

# 2. Define the Neural Network class

class SimpleNeuralNetwork:
  def __init__(self, input_size, hidden_size, output_size, learning_rate=0.1):
    self.input_size = input_size
    self.hidden_size = hidden_size
    self.output_size = output_size
```

```
        self.learning_rate = learning_rate

        # Initialize weights and biases randomly
        self.W1 = np.random.uniform(size=(self.input_size, self.hidden_size))
        self.b1 = np.random.uniform(size=(1, self.hidden_size))
        self.W2 = np.random.uniform(size=(self.hidden_size, self.output_size))
        self.b2 = np.random.uniform(size=(1, self.output_size))

    def forward_propagation(self, X):
        """Performs the forward pass of the network."""
        self.hidden_input = np.dot(X, self.W1) + self.b1
        self.hidden_output = sigmoid(self.hidden_input)
        self.output = np.dot(self.hidden_output, self.W2) + self.b2
        self.predicted_output = sigmoid(self.output)
        return self.predicted_output

    def backward_propagation(self, X, y, predicted_output):
        """Performs the backward pass (backpropagation)."""
        # Calculate output layer error
        output_error = y - predicted_output
        output_delta = output_error * sigmoid_derivative(predicted_output)

        # Calculate hidden layer error
        hidden_error = output_delta.dot(self.W2.T)
        hidden_delta = hidden_error * sigmoid_derivative(self.hidden_output)

        # Update weights and biases
        self.W2 += self.hidden_output.T.dot(output_delta) * self.learning_rate
        self.b2 += np.sum(output_delta, axis=0, keepdims=True) * self.learning_rate
        self.W1 += X.T.dot(hidden_delta) * self.learning_rate
        self.b1 += np.sum(hidden_delta, axis=0, keepdims=True) * self.learning_rate
```

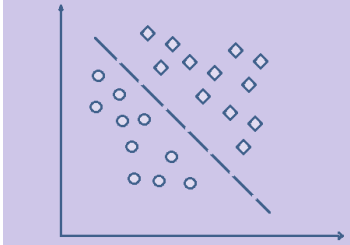
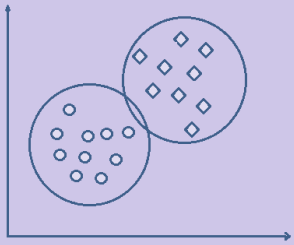
```
  def train(self, X, y, epochs):
    """Trains the neural network."""
    for i in range(epochs):
      predicted_output = self.forward_propagation(X)
      self.backward_propagation(X, y, predicted_output)
      loss = mean_squared_error(predicted_output, y)
      if i % 100 == 0:
        print(f"Epoch {i}, Loss: {loss:.4f}")

# 3. Prepare the XOR dataset

# Input features
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)

# Target output
y_xor = np.array([[0], [1], [1], [0]], dtype=float)

# 4. Create and train the network

# Normalize input features if necessary (though XOR is simple, good practice)
X_xor_scaled = X_xor / np.amax(X_xor, axis=0)
y_xor_scaled = y_xor / 1 #  Already in the 0-1 range

# Instantiate the neural network
nn = SimpleNeuralNetwork(input_size=2, hidden_size=2, output_size=1, learning_rate=0.5)

# Train the network
epochs = 5000
nn.train(X_xor_scaled, y_xor_scaled, epochs)

# 5. Test the network
```

```python
# 5. Test the network

print("\nTesting the trained network:")
for inputs, true_output in zip(X_xor_scaled, y_xor_scaled):
  predicted = nn.forward_propagation(inputs)
  print(f"Input: {inputs}, Predicted Output: {predicted[0]:.4f}, True Output: {true_output[0]}")
```

```
Epoch 0, Loss: 0.3232
Epoch 100, Loss: 0.2499
Epoch 200, Loss: 0.2499
Epoch 300, Loss: 0.2498
Epoch 400, Loss: 0.2497
Epoch 500, Loss: 0.2493
Epoch 600, Loss: 0.2486
Epoch 700, Loss: 0.2463
Epoch 800, Loss: 0.2396
Epoch 900, Loss: 0.2234
Epoch 1000, Loss: 0.2005
Epoch 1100, Loss: 0.1822
Epoch 1200, Loss: 0.1653
Epoch 1300, Loss: 0.1331
Epoch 1400, Loss: 0.0718
Epoch 1500, Loss: 0.0344
Epoch 1600, Loss: 0.0198
Epoch 1700, Loss: 0.0132
Epoch 1800, Loss: 0.0097
Epoch 1900, Loss: 0.0076
Epoch 2000, Loss: 0.0062
Epoch 2100, Loss: 0.0052
Epoch 2200, Loss: 0.0044
Epoch 2300, Loss: 0.0039
Epoch 2400, Loss: 0.0034
Epoch 2500, Loss: 0.0031
Epoch 2600, Loss: 0.0028
Epoch 2700, Loss: 0.0026
```

Viva-Questions

1	**What is machine learning?**
	Machine learning (ML) is a type of artificial intelligence (AI) that allows software applications to become more accurate at predicting outcomes without being explicitly programmed to do so. Machine learning algorithm use historical data as input to predict new output values.
2	**Features of Machine Learning** i. It is a data-driven technology. ii. It can learn from past data and improve automatically. iii. It uses data to detect various patterns in a given dataset. iv. It is much similar to data mining as it also deals with the huge amount of the data.
3	**Types of Machine Learning** 1. Supervised learning 2. Unsupervised learning 3. Semi supervised learning 4. Reinforcement learning 1. Supervised learning: Supervised learning is a type of machine learning method in which we provide sample labeled data to the machine learning system in order to train it, and on the basis, it predicts the output. The goal of supervised learning is to map input data with the output data. Supervised learning can be grouped further into two categories pf algorithms: i. Classification ii. Regression Classification: Classification algorithms are used to solve the classification problems in which the output variable is categorical, such as Yes or No, Male or Female etc. The classification algorithms predict the categories present in the dataset. *Real world examples: • Spam Detection • Email filtering Classification algorithms: - 1. Random Forest Algorithms 2. Decision Tree Algorithms 3. Logistic Regression Algorithm 4. Support Vector Machine Algorithms iii.

4	**What is the difference between clustering and classification?** **Classification** / **Clustering** Process of classifying the data with help of class lable / It is similar to classification but there are no predefined class lable A supervised learning technique / An unsupervised learning technique Goal of assigning new input to a class / Goal of finding similarities within a given dataset Works with labeled data / Works with unlabeled data Known number of classes / Unknown number of classes  
5	**What is Supervised, Unsupervised and reinforcement learning?** Supervised-labelled does classification unsupervised-not labeled finds similarity reinforcement-continous learning by reward and penalty.
6	**What is centroid in kmeans and how do u determine?** Centroid- imaginary center of cluster By averaging the data points feature wise
7	**Is KNN a classification or clustering?** KNN is a supervised learning algorithm used for classification.
8.	**8.Define supervised learning**
9.	**Define unsupervised learning**
10.	**Define semi supervised learning**
11.	**Define reinforcement learning**
12.	**What do you mean by hypotheses.**

13.	**What is classification?**
14.	**What is clustering?**
15.	**Define precision, accuracy and recall.**
16.	**Define entropy.**
17.	**Define regression.**
18.	**How Knn is different from k-means clustering?**
19.	**What is concept learning?**
20.	**Define specific boundary and general boundary**
21.	**Define target function**
22.	**Define decision tree**
23.	**What is ANN?**
24.	**Explain gradient descent approximation**
25.	**State Bayes theorem**
26.	**Define Bayesian belief networks**
27	**Differentiate hard and soft clustering**
28	**Why Use FIND-S Algorithm?** 1. Understanding Hypothesis Space * FIND-S helps beginners understand how a machine can form hypotheses and generalize from data. * It operates in a version space, narrowing down to the most specific hypothesis that matches all positive examples. 2. Foundational Learning Algorithm * It introduces the idea of inductive learning, where we learn general rules from specific examples.

	* Teaches important ML concepts: attributes, instances, target function, and hypothesis generalization. 3. Efficiency in Simpler Tasks * Very efficient for domains where: * Only positive examples are available. * The target concept is consistent and deterministic.Best Used When: * You're teaching/learning about concept learning. * The data has clear, noise-free positive examples. * You need a simple, explainable model for educational or prototyping purposes
29	**What is Overfitting?** **When a model learns the training data too well, including noise, and performs poorly on new data.**
30	**How can you prevent overfitting?** **Techniques like regularization, cross-validation, and using more data.**
31	**What is K-Nearest Neighbors (KNN)?** **A simple algorithm that classifies data points based on the majority class of their k nearest neighbors.**
32	**What is a Decision Tree?** **A tree-like structure that makes decisions based on a series of questions about the data.**
33	**What is Naive Bayes?** **A probabilistic classifier that assumes independence between features**
34	**What is Linear Regression?** **A method for predicting a continuous target variable based on a linear relationship with one or more predictor variables.**
35	**What is K-Means Clustering?** **An algorithm that partitions data into k clusters based on similarity.**
36	**What is PCA (Principal Component Analysis)?** **A dimensionality reduction technique used to reduce the number of variables in a dataset while retaining most of the variance.**
37	**What are the different types of Neural Networks? Feedforward, convolutional, recurrent.**
38	**What is a Confusion Matrix?** **A table that summarizes the performance of a classification model.**
39	**What is Precision, Recall, and F1-score?** **Metrics used to evaluate the performance of classification models.**
40	**What is Accuracy?** **The overall correctness of a model's predictions.**
41	**What is Cross-Validation?**

	A technique for evaluating a model's performance by splitting the data into multiple folds and training and testing on different combinations.
42	**How do you evaluate the performance of a clustering algorithm?** **Using metrics like silhouette score, Davies-Bouldin index, or visual inspection**
43	**What is data preprocessing?** **Cleaning and transforming raw data into a suitable format for machine learning models**
44	**How do you handle missing data? Techniques like imputation (mean, median, mode, or more sophisticated methods) or removing rows/columns with missing values.**
45	**What are some common Python libraries for Machine Learning?** **NumPy, Pandas, Scikit-learn, TensorFlow, Keras, etc.**
46	**How would you implement cross-validation in Python?** **Using cross_val_score or KFold from scikit-learn.**
47	**How do you tune hyperparameters in Python?** **Techniques like grid search, random search, or Bayesian optimization.**
48	**What Is Normalization, And Why Is It An Important Preprocessing Step In Machine Learning?** **Normalization scales numerical data to a standard range, improving model performance and convergence speed. It ensures that features with different units do not dominate the learning process.**
49	**Can You Explain The Difference Between Precision And Recall, And When Would You Use Each Metric?** **Precision and recall evaluate classification performance, especially in imbalanced datasets. Precision measures how many predicted positives are correct, while recall shows how many actual positives were detected.** **Below is a comparison of precision and recall:**

Aspect	Precision	Recall
Definition	Ratio of correctly predicted positives to total predicted positives	Ratio of correctly predicted positives to actual positives
Use Case	When false positives must be minimized (e.g., spam detection)	When false negatives must be minimized (e.g., disease detection)
Formula	TP / (TP + FP)	TP / (TP + FN)
Focus	Accuracy of positive predictions	Capturing all actual positives
Trade-off	Higher precision reduces recall	Higher recall reduces precision

50	**What is Principal Component Analysis (PCA), And When Should It Be Used?** **PCA reduces the dimensionality of datasets while preserving important information. It transforms correlated features into uncorrelated principal components.** **Below are situations where PCA is useful:** • **High-Dimensional Data – Reduces features while maintaining variance.** • **Noise Reduction – Eliminates redundant information in datasets.** • **Improves Model Performance – Speeds up training by reducing complexity.** • **Visualization – Helps in 2D or 3D representation of high-dimensional data.** • **Example – A facial recognition system uses PCA to extract essential features like nose and eye structure.**
51	**What Are The Key Differences Between Manhattan Distance And Euclidean Distance, And When Is Each One Preferred?** **Distance metrics measure similarity between data points in machine learning models. Manhattan and Euclidean distances are commonly used.** **Below is a comparison of both:** **Aspect** / **Manhattan Distance** / **Euclidean Distance** Definition / Measures distance along axes / Measures straight-line distance Formula / Sum of absolute differences / Square root of squared differences Use Case / Grid-based movements (e.g., chess, city blocks) / Continuous space (e.g., clustering, regression) Computational Cost / Lower, simpler calculations / Higher due to square root computation Example / Delivery routes in a city grid / Distance between two GPS points
52	**How Do You Interpret A Confusion Matrix To Evaluate A Machine Learning Model?** **A confusion matrix evaluates classification performance by comparing predicted and actual values. It consists of four key components:** **Below are the components of a confusion matrix:** • **True Positives (TP) – Correctly predicted positive cases.** • **True Negatives (TN) – Correctly predicted negative cases.** • **False Positives (FP) – Incorrectly predicted positive cases.** • **False Negatives (FN) – Incorrectly predicted negative cases.** **Example: In a fraud detection system, if 90 frauds are correctly detected (TP), 10 frauds go undetected (FN), 5 normal transactions are flagged as fraud (FP), and 95 normal transactions are correctly classified (TN), then precision and recall can be calculated.**
53	**What Is The Purpose Of Splitting A Dataset Into Training And Validation Sets, And How Does It Help Model Evaluation?**

Splitting datasets ensures proper model evaluation and prevents overfitting. The training set helps the model learn, while the validation set assesses performance.
Here's why dataset splitting is essential:

- **Prevents Overfitting – Ensures the model does not memorize training data.**
- **Improves Generalization – Helps test model performance on unseen data.**
- **Allows Hyperparameter Tuning – Helps adjust learning rates, tree depths, etc.**
- **Used in Cross-Validation – Further improves model selection.**
- **Example – A handwriting recognition model is trained on 80% of images and validated on the remaining 20%.**

54 **What Is The Primary Difference Between k-means Clustering And The KNN Algorithm?**
Both k-means clustering and the KNN algorithm are used for machine learning but serve different purposes.
Below is a comparison of both:

Aspect	k-means Clustering	KNN Algorithm
Type	Unsupervised Learning	Supervised Learning
Purpose	Groups similar data into clusters	Classifies new data points
Input Required	Unlabeled data	Labeled training data
Algorithm Basis	Iterative centroid optimization	Distance-based classification
Example	Customer segmentation	Spam email detection

55 **What Are Some Common Techniques To Visualize High-Dimensional Data In Two-Dimensional Space?**
High-dimensional data is difficult to interpret, so dimensionality reduction techniques help visualize it effectively.
Below are some common methods:

- **Principal Component Analysis (PCA) – Reduces dimensions while preserving variance.**
- **t-Distributed Stochastic Neighbor Embedding (t-SNE) – Captures complex relationships for clustering.**

56 **Why Is The Curse Of Dimensionality A Challenge In Machine Learning, And How Can It Be Mitigated?**
The curse of dimensionality occurs when increasing features negatively impacts model performance.
Here's why it is a challenge:

- **Sparse Data – Higher dimensions cause data points to spread out, reducing meaningful relationships.**
- **Computational Cost – More dimensions require higher processing power.**
- **Overfitting Risk – Too many features cause models to learn noise.**

Below are techniques to mitigate it:

- **Feature Selection – Retains only relevant variables.**
- **Dimensionality Reduction – Uses PCA or t-SNE to reduce features.**
- **Example – A text classification model with 10,000 features benefits from feature selection.**

57 **Which Regression Metric (MAE, MSE, or RMSE) Is Most Resistant To Outliers, And Why?**

Regression models use different metrics to evaluate error, and some handle outliers better than others.

Below is a comparison:

Metric	Sensitivity to Outliers	Explanation
MAE (Mean Absolute Error)	Low	Uses absolute differences, making it more stable against outliers.
MSE (Mean Squared Error)	High	Squares differences, increasing the effect of outliers.
RMSE (Root Mean Squared Error)	High	Similar to MSE but takes the square root for better interpretation.

58 **What is Linear Discriminant Analysis (LDA), And When Is It Used In Machine Learning?**

Linear Discriminant Analysis (LDA) reduces dimensionality while preserving class separability. It is widely used for classification.

Below are key applications of LDA:

- **Feature Reduction – Reduces high-dimensional data while maintaining class separability.**
- **Pattern Recognition – Used in facial recognition systems.**
- **Spam Detection – Helps classify emails as spam or not spam.**
- **Medical Diagnosis – Identifies diseases based on patient data.**

Example: In image classification, LDA projects high-dimensional images onto a lower-dimensional space, improving classification accuracy.

59 **What Are The Main Advantages And Disadvantages Of Decision Tree-Based Models In Machine Learning?**

Decision trees offer simplicity but have limitations.

Below is a comparison of advantages and disadvantages:

Aspect	Advantages	Disadvantages
Interpretability	**Easy to understand**	**Complex trees are hard to interpret**
Overfitting	**Performs well on training data**	**Overfits with deep trees**

	Computational Cost	**Fast training speed**	**Slower with large datasets**
	Flexibility	**Works for classification & regression**	**Sensitive to small data changes**
	Handling Outliers	**Handles outliers well**	**Can be biased toward majority class**

60 **How Would You Evaluate The Performance Of A Linear Regression Model, And Which Metrics Do You Consider Most Critical?**

Evaluating a linear regression model ensures it generalizes well.

Below are critical evaluation metrics:

- **Mean Absolute Error (MAE) – Measures average absolute difference between actual and predicted values.**
- **Mean Squared Error (MSE) – Penalizes larger errors more heavily.**
- **Root Mean Squared Error (RMSE) – Square root of MSE; useful for large error sensitivity.**
- **R-squared (R^2) – Explains variance in the target variable.**
- **Adjusted R^2 – Adjusts for the number of predictors to avoid overfitting.**

Example: In a house price prediction model, a low RMSE and high R^2 indicate a good fit.